04 Annotation validation

Independent checks on the assigned cell-type labels from notebook 03 include: 1. Coverage (resolved vs unresolved). 2. Enrichment of the scoring markers in their assigned class (one-sided Wilcoxon, BH). 3. Concordance with a held-out marker panel that shares no genes with the annotation. 4. Association of scores with sequencing depth 5. Agreement with three external references via SingleR (Siletti 2023 adult human brain; a developing human brain reference; Allen Smart-seq2 MTG). - This step is compute intensive and was performed on the cluster - Analysis included in the notebook uses precomputed predictions deposited on Zenodo - To rerun SingleR on the reference objects listed in `config/paths.example.yaml`, set `run_singler = TRUE`.

```
suppressPackageStartupMessages({
  library(SingleCellExperiment)
  library(Matrix)
})
source(file.path("..", "scripts", "helpers.R"))
paths <- load_paths()
```

Parameters

```
qc_unresolved  <- c("Unknown", "Neuron_unresolved", "NonNeuron_unresolved")
ann_unresolved <- c("Neuron_unresolved", "NonNeuron_unresolved")

# Marker enrichment
pseudocount <- 0.1
fdr_cutoff <- 0.05
min_cells_per_side <- 3L

# Held-out panel (must not overlap annotation genes)
heldout_panel <- list(
  Neuron_core = c("CORO6", "SLC12A5"),
  Excitatory_neurons = c("CBLN2", "CAMK2A"),
  Inhibitory_neurons = c("GABRB2", "GRIK2"),
  Oligodendrocytes = c("UGT8", "MAL"),
  OPCs = c("BCAN"),
  Astrocytes = c("FGFR3", "AGT"),
  Microglia = c("DOCK8", "PTPRC"),
  Pericytes = c("GUCY1A2", "COLEC12"),
  Endothelial = c("FMO2", "LAMA5"))

heldout_min_cells_per_class <- 2L

# SingleR
canonical_classes <- c("Excitatory_neurons", "Inhibitory_neurons", "Astrocytes", "OPCs",
                       "Oligodendrocytes", "Microglia", "Endothelial", "Pericytes")

out_of_scope_labels   <- c("neural_progenitor", "smooth_muscle_cell")

gene_exclude_patterns <- c("^MT-", "^RPL", "^RPS")
```

```
prune_nmads <- 3L

references <- list(
  siletti = list(path = paths$ref_siletti, subset_field = "data_type",
                       subset_value = "siletti", label_field = "cluster_annotation"),
  developmental = list(path = paths$ref_developmental, subset_field = "data_type",
                       subset_value = "dev", label_field = "cluster_annotation"),
  ss2 = list(path = paths$ref_ss2, subset_field = NA, subset_value = NA,
                       label_field = "subclass_label"))
```

```
sce <- readRDS(file.path(paths$work_dir, "03_annotated.rds"))
per_cell <- read.delim(file.path(paths$work_dir, "03_annotation_per_cell.tsv"),
 ↪  stringsAsFactors = FALSE)

stopifnot(identical(per_cell$cell_id, colnames(sce)))
final <- per_cell$final_celltype_label
log_sym <- symbol_matrices(sce, "symbol")$logcounts
is_res <- !(final %in% qc_unresolved)

# Scoring markers and EX/IN genes saved by notebook 03

markers <- readRDS(file.path(paths$work_dir, "03_marker_lists.rds"))
known_list_syms <- markers$known_list_syms

annotation_used_genes <- unique(c(unlist(known_list_syms),
 ↪  unlist(markers$exin_evidence)))
```

1. Coverage

```
coverage <- data.frame(
  metric = c("n_total", "n_resolved", "n_unresolved", "fraction_resolved",
   ↪  "fraction_unresolved"),
  value  = c(length(final), sum(is_res), sum(!is_res), mean(is_res), mean(!is_res)))
coverage
```

```
##                metric        value
## 1             n_total 1050.0000000
## 2          n_resolved  852.0000000
## 3        n_unresolved  198.0000000
## 4   fraction_resolved    0.8114286
## 5 fraction_unresolved    0.1885714
```

```
table(final_label = final)
```

```
## final_label
##           Astrocytes          Endothelial   Excitatory_neurons
##                   52                    4                  318
##   Inhibitory_neurons            Microglia    Neuron_unresolved
##                  128                   37                  175
## NonNeuron_unresolved     Oligodendrocytes                 OPCs
##                   23                  226                   82
##            Pericytes
##                    5
```

2. Scoring-marker enrichment

For each class and each of its markers: a one-sided Wilcoxon test comparing resolved nuclei of that class against all other resolved nuclei, with BH correction within the class. A marker is enriched when FDR $<$ 0.05 and log2FC $>$ 0.

```
enrichment <- do.call(rbind, lapply(names(known_list_syms), function(ct) {
  markers <- intersect(known_list_syms[[ct]], rownames(log_sym))
  in_cl  <- which(final == ct & is_res)
  out_cl <- which(final != ct & is_res)
  if (length(markers) == 0L || length(in_cl) < min_cells_per_side ||
      length(out_cl) < min_cells_per_side) return(NULL)
  rows <- do.call(rbind, lapply(markers, function(g) {
    xi <- as.numeric(log_sym[g, in_cl]); xo <- as.numeric(log_sym[g, out_cl])
    wt <- suppressWarnings(stats::wilcox.test(xi, xo, alternative = "greater"))
    data.frame(cell_class = ct, gene = g, n_in = length(in_cl), n_out = length(out_cl),
               mean_in = mean(xi), mean_out = mean(xo),
               log2FC = log2((mean(xi) + pseudocount) / (mean(xo) + pseudocount)),
               p_value = wt$p.value, stringsAsFactors = FALSE)
  }))
  rows$FDR <- stats::p.adjust(rows$p_value, method = "BH")
  rows$enriched <- rows$FDR < fdr_cutoff & rows$log2FC > 0
  rows
}))
aggregate(enriched ~ cell_class, data = enrichment, FUN = mean)
```

```
##           cell_class  enriched
## 1         Astrocytes 1.0000000
## 2        Endothelial 0.2222222
## 3 Excitatory_neurons 0.7272727
## 4 Inhibitory_neurons 0.6666667
## 5          Microglia 0.9000000
## 6   Oligodendrocytes 1.0000000
## 7               OPCs 1.0000000
## 8          Pericytes 0.5000000
```

3. Held-out marker panel

For each held-out gene, median logcounts are compared across resolved classes, using only classes with at least 2 nuclei. The gene validates its expected class when that class has the unique highest median.

```
stopifnot(length(intersect(unlist(heldout_panel), annotation_used_genes)) == 0L)
class_labs <- setdiff(unique(final[is_res]), qc_unresolved)

heldout <- do.call(rbind, lapply(names(heldout_panel), function(ct) {
  do.call(rbind, lapply(heldout_panel[[ct]], function(g) {
    status <- "missing_gene"; maxers <- NA_character_
    if (g %in% rownames(log_sym)) {
      med <- vapply(class_labs, function(k) {
        idx <- which(final == k & is_res)
        if (length(idx) < heldout_min_cells_per_class) return(NA_real_)
        stats::median(as.numeric(log_sym[g, idx]))
      }, numeric(1))
      med <- med[!is.na(med)]
      if (length(med) == 0L || !(ct %in% names(med))) {
        status <- "insufficient_class_support"
```

```
      } else {
        maxers <- names(med)[med == max(med)]
        status <- if (length(maxers) > 1L) "tie_for_max" else
          if (identical(maxers, ct)) "correct_unique_max" else "incorrect_unique_max"
        maxers <- paste(maxers, collapse = ";")
      }
    }
    data.frame(expected_class = ct, gene = g, maximizing_classes = maxers,
               validation_status = status, stringsAsFactors = FALSE)
  }))
}))

heldout_per_class <- do.call(rbind, lapply(split(heldout, heldout$expected_class),
 ↪  function(df) {
  ev <- df$validation_status %in% c("correct_unique_max", "incorrect_unique_max",
 ↪  "tie_for_max")
  data.frame(cell_class = df$expected_class[1], n_evaluable = sum(ev),
             n_correct = sum(df$validation_status == "correct_unique_max"),
             accuracy_among_evaluable = if (any(ev))
               sum(df$validation_status == "correct_unique_max") / sum(ev) else NA_real_)
}))

heldout_per_class
```

```
##                            cell_class n_evaluable n_correct
## Astrocytes                 Astrocytes           2         2
## Endothelial               Endothelial           2         0
## Excitatory_neurons Excitatory_neurons           2         0
## Inhibitory_neurons Inhibitory_neurons           2         2
## Microglia                   Microglia           2         2
## Neuron_core               Neuron_core           0         0
## Oligodendrocytes     Oligodendrocytes           2         2
## OPCs                             OPCs           1         1
## Pericytes                   Pericytes           2         2
##                    accuracy_among_evaluable
## Astrocytes                                1
## Endothelial                               0
## Excitatory_neurons                        0
## Inhibitory_neurons                        1
## Microglia                                 1
## Neuron_core                              NA
## Oligodendrocytes                          1
## OPCs                                      1
## Pericytes                                 1
```

4. Score-depth associations

```
counts <- assay(sce, "counts")
library_size <- Matrix::colSums(counts)
detected     <- Matrix::colSums(counts > 0)

assoc <- function(x, y, pair, subset) {
  ok <- is.finite(x) & is.finite(y)
  if (sum(ok) < 5L || stats::sd(x[ok]) == 0 || stats::sd(y[ok]) == 0)
```

```
    return(data.frame(metric_pair = pair, subset = subset, n = sum(ok),
                      pearson = NA_real_, spearman = NA_real_))
  data.frame(metric_pair = pair, subset = subset, n = sum(ok),
             pearson  = stats::cor(x[ok], y[ok], method = "pearson"),
             spearman = stats::cor(x[ok], y[ok], method = "spearman"))
}

max_exin <- suppressWarnings(pmax(per_cell$marker_contrast_Excitatory_neurons,
                                  per_cell$marker_contrast_Inhibitory_neurons, na.rm =
 ↪  TRUE))

max_exin[!is.finite(max_exin)] <- NA_real_

score_depth <- rbind(
  assoc(library_size, per_cell$neuron_core_marker_contrast,
   ↪  "library_size~neuron_core_contrast", "all"),
  assoc(detected, per_cell$neuron_core_marker_contrast,
   ↪  "detected_features~neuron_core_contrast", "all"),
  assoc(library_size, max_exin, "library_size~max_exin_contrast", "all"),
  assoc(library_size, per_cell$exin_margin, "library_size~exin_margin", "all"),
  assoc(library_size, per_cell$nonneuron_top1_score, "library_size~nonneuron_top1_score",
   ↪  "nonneuron"),
  assoc(library_size, per_cell$nonneuron_score_margin,
   ↪  "library_size~nonneuron_score_margin", "nonneuron"),
  assoc(library_size, per_cell$nonneuron_predicted_detection_fraction,
        "library_size~nonneuron_detection_fraction", "nonneuron"))

score_depth
```

```
##                                 metric_pair    subset    n     pearson
## 1         library_size~neuron_core_contrast       all 1050  0.35547434
## 2    detected_features~neuron_core_contrast       all 1050  0.52142579
## 3            library_size~max_exin_contrast       all 1050 -0.04951446
## 4                  library_size~exin_margin       all 1035  0.37785920
## 5         library_size~nonneuron_top1_score nonneuron  429  0.42550302
## 6       library_size~nonneuron_score_margin nonneuron  429  0.33236816
## 7 library_size~nonneuron_detection_fraction nonneuron  429  0.34111586
##     spearman
## 1 0.21914949
## 2 0.48074537
## 3 0.01358508
## 4 0.31319591
## 5 0.55635443
## 6 0.46809496
## 7 0.48875251
```

5. Reference agreement (SingleR)

Reference labels are mapped to broad classes. Genes matching `^MT-`, `^RPL` or `^RPS` are removed from both the query and the reference. Labels are pruned at 3 MADs, and the pruned label is used as the reference call.

```
broad_label_groups <- list(
  Excitatory_neurons = c("Glut", "Glutamatergic", "Superficial_layer_glutamatergic",
    "Deep_layer_glutamatergic", "L2/3 IT", "L4 IT", "L6 CT", "L5 IT", "L6 IT", "L6 IT
     ↪  Car3",
```

```
    "L5ET", "L6b", "L5/6 NP", "L5 ET"),
  Inhibitory_neurons = c("PVALB", "SST", "LAMP5", "PAX6", "VIP", "GABAergic",
    "caudal ganglionic eminence derived interneuron",
    "medial ganglionic eminence derived interneuron", "MGE", "CGE"),
  Endothelial = c("Endothelial", "endothelial cell"),
  Pericytes = c("Fibroblast_pericyte", "VLMC", "fibroblast", "pericyte"),
  Microglia = c("CNS_macrophage", "Microglia", "central nervous system macrophage",
   ↪  "microglia"),
  Astrocytes = c("Astrocyte", "astrocyte", "Astrocytes"),
  Oligodendrocytes = c("Oligo", "Oligodendrocyte", "oligodendrocyte",
   ↪  "Oligodendrocytes"),
  OPCs = c("OPCs", "OPC", "oligodendrocyte precursor cell"),
  neural_progenitor = c("radial glial cell", "neural progenitor cell", "neuron",
   ↪  "GABAergic_NPC"),
  smooth_muscle_cell = c("vascular associated smooth muscle cell", "smooth muscle cell"))

make_broad_label <- function(x) {
  x <- as.character(x); out <- x; assigned <- rep(FALSE, length(x))
  for (b in names(broad_label_groups)) {
    hit <- !assigned & x %in% broad_label_groups[[b]]
    out[hit] <- b; assigned <- assigned | hit
  }
  out
}
```

```
suppressPackageStartupMessages(library(SingleR))

run_singler <- function(name, ref) {
  obj <- readRDS(ref$path)
  if (methods::is(obj, "Seurat")) {
    if (!is.na(ref$subset_field))
      obj <- subset(obj, cells =
 ↪  rownames(obj@meta.data)[obj@meta.data[[ref$subset_field]] == ref$subset_value])
    obj <- Seurat::as.SingleCellExperiment(obj)
  } else if (!is.na(ref$subset_field)) {
    obj <- obj[, which(obj[[ref$subset_field]] == ref$subset_value)]
  }

  if (!("logcounts" %in% assayNames(obj)))
    obj <- scuttle::logNormCounts(obj, transform = "log", size.factors = NULL, name =
 ↪  "logcounts")
  common <- intersect(rownames(obj), rownames(sce))
  keep <- common[!Reduce(`|`, lapply(gene_exclude_patterns, grepl, x = common))]

  broad <- make_broad_label(obj[[ref$label_field]])
  pred <- SingleR::SingleR(test = assay(sce[keep, ], "logcounts"),
                           ref = assay(obj[keep, ], "logcounts"), labels = broad)

  pruned_flag <- SingleR::pruneScores(pred, nmads = prune_nmads)
  unpruned <- as.character(pred$labels)
  pruned <- unpruned; pruned[!is.na(pruned_flag) & pruned_flag] <- NA_character_
  status <- ifelse((!is.na(pruned_flag) & pruned_flag) | is.na(unpruned), "pruned_to_na",
            ifelse(unpruned %in% out_of_scope_labels, "reference_out_of_scope",
            ifelse(unpruned %in% canonical_classes, "mapped_pruned",
             ↪  "unmapped_reference_label")))
```

```
  data.frame(cell_id = colnames(sce), reference_name = name,
             ref_label_unpruned = unpruned, ref_label_pruned = pruned,
             ref_delta_next = as.numeric(pred$delta.next), reference_status = status,
             stringsAsFactors = FALSE)
}

for (nm in names(references))
  save_tsv(run_singler(nm, references[[nm]]),
           file.path(paths$work_dir, "04_singler", paste0("reference_", nm, ".tsv")))
```

```
pred_dir <- if (isTRUE(params$run_singler)) file.path(paths$work_dir, "04_singler") else
  file.path(paths$zenodo_root, "metadata", "reference_annotations")

cohens_kappa <- function(a, b) {
  n <- length(a); if (n < 2L) return(NA_real_)
  lv <- sort(unique(c(a, b)))
  pe <- sum(as.numeric(table(factor(a, lv)) / n) * as.numeric(table(factor(b, lv)) / n))
  if (isTRUE(all.equal(pe, 1))) return(NA_real_)
  (mean(a == b) - pe) / (1 - pe)
}

per_class_directional <- function(pred, ref, classes) {
  do.call(rbind, lapply(classes, function(k) {
    n_ref <- sum(ref == k); n_pred <- sum(pred == k); n_hit <- sum(ref == k & pred == k)
    recall <- precision <- f1 <- NA_real_
    if (n_ref > 0L) {
      recall <- n_hit / n_ref
      precision <- if (n_pred > 0L) n_hit / n_pred else 0
      f1 <- if (recall + precision > 0) 2 * precision * recall / (precision + recall)
 ↪  else 0
    }
    data.frame(class = k, n_reference = n_ref, n_predicted = n_pred, n_agree = n_hit,
               recall = recall, precision = precision, f1 = f1)
  }))
}

agreement <- do.call(rbind, lapply(names(references), function(nm) {
  pred <- utils::read.table(file.path(pred_dir, paste0("reference_", nm, ".tsv")),
                            header = TRUE, sep = "\t", quote = "", comment.char = "",
                            stringsAsFactors = FALSE, fill = TRUE, na.strings = c("NA",
                             ↪  ""))
  stopifnot(!anyDuplicated(pred$cell_id), setequal(pred$cell_id, colnames(sce)))

  refp <- pred$ref_label_pruned[match(colnames(sce), pred$cell_id)]
  space <- !(final %in% ann_unresolved) & !is.na(refp) &
           !(refp %in% out_of_scope_labels) & refp %in% canonical_classes
  pc <- per_class_directional(final[space], refp[space], canonical_classes)
  sup <- pc$n_reference > 0L

  data.frame(reference_name = nm, total_query_cells = length(final),
             n_in_agreement_space = sum(space),
             all_cells_agreement = mean(final == ifelse(is.na(refp), "pruned_NA", refp)),
             restricted_agreement = mean(final[space] == refp[space]),
             cohens_kappa = cohens_kappa(final[space], refp[space]),
```

```
            macro_recall = mean(pc$recall[sup]), macro_precision =
             ↪  mean(pc$precision[sup]),
            macro_f1 = mean(pc$f1[sup]))
}))
agreement
```

```
##   reference_name total_query_cells n_in_agreement_space all_cells_agreement
## 1        siletti              1050                  850           0.6028571
## 2  developmental              1050                  607           0.4971429
## 3            ss2              1050                  852           0.5904762
##   restricted_agreement cohens_kappa macro_recall macro_precision  macro_f1
## 1            0.7447059    0.6611132    0.7477950       0.7107434 0.7011403
## 2            0.8599671    0.8182321    0.9186386       0.8507424 0.8651600
## 3            0.7276995    0.6418144    0.7727515       0.6385650 0.6506925
```

```
out <- file.path(paths$work_dir, "04_annotation_validation")
save_tsv(coverage, file.path(out, "coverage.tsv"))
save_tsv(enrichment, file.path(out, "marker_enrichment_per_gene.tsv"))
save_tsv(heldout, file.path(out, "heldout_validation_per_gene.tsv"))
save_tsv(heldout_per_class, file.path(out, "heldout_validation.tsv"))
save_tsv(score_depth, file.path(out, "score_depth.tsv"))
save_tsv(agreement, file.path(out, "reference_agreement_summary.tsv"))
sessionInfo()
```

```
## R version 4.5.2 (2025-10-31)
## Platform: aarch64-apple-darwin20
## Running under: macOS Tahoe 26.5.2
##
## Matrix products: default
## BLAS:   /System/Library/Frameworks/Accelerate.framework/Versions/A/Frameworks/vecLib.
 ↪  framework/Versions/A/libBLAS.dylib
## LAPACK:
 ↪  /Library/Frameworks/R.framework/Versions/4.5-arm64/Resources/lib/libRlapack.dylib;
 ↪  LAPACK version 3.12.1
##
## locale:
## [1] C.UTF-8/C.UTF-8/C.UTF-8/C/C.UTF-8/C.UTF-8
##
## time zone: America/New_York
## tzcode source: internal
##
## attached base packages:
## [1] stats4    stats     graphics  grDevices utils     datasets  methods
## [8] base
##
## other attached packages:
##  [1] igraph_2.2.1                scater_1.38.0
##  [3] ggplot2_4.0.1               scran_1.38.0
##  [5] Matrix_1.7-4                scuttle_1.20.0
##  [7] SingleCellExperiment_1.32.0 SummarizedExperiment_1.40.0
##  [9] Biobase_2.70.0              GenomicRanges_1.62.1
## [11] Seqinfo_1.0.0               IRanges_2.44.0
## [13] S4Vectors_0.48.0            BiocGenerics_0.56.0
## [15] generics_0.1.4              MatrixGenerics_1.22.0
## [17] matrixStats_1.5.0
```

```
##
## loaded via a namespace (and not attached):
##  [1] beeswarm_0.4.0      gtable_0.3.6        xfun_0.55
##  [4] ggrepel_0.9.6       lattice_0.22-7      vctrs_0.7.1
##  [7] tools_4.5.2         parallel_4.5.2      tibble_3.3.1
## [10] cluster_2.1.8.2     pkgconfig_2.0.3     BiocNeighbors_2.4.0
## [13] RColorBrewer_1.1-3  S7_0.2.1            dqrng_0.4.1
## [16] lifecycle_1.0.5     compiler_4.5.2      farver_2.1.2
## [19] statmod_1.5.1       tinytex_0.58        bluster_1.20.0
## [22] codetools_0.2-20    vipor_0.4.7         htmltools_0.5.9
## [25] yaml_2.3.12         pillar_1.11.1       BiocParallel_1.44.0
## [28] DelayedArray_0.36.0 limma_3.66.0        viridis_0.6.5
## [31] abind_1.4-8         tidyselect_1.2.1    rsvd_1.0.5
## [34] locfit_1.5-9.12     metapod_1.18.0      digest_0.6.39
## [37] BiocSingular_1.26.0 dplyr_1.2.0         fastmap_1.2.0
## [40] grid_4.5.2          cli_3.6.5           SparseArray_1.10.8
## [43] magrittr_2.0.4      S4Arrays_1.10.1     withr_3.0.2
## [46] edgeR_4.8.2         scales_1.4.0        ggbeeswarm_0.7.3
## [49] rmarkdown_2.30      XVector_0.50.0      otel_0.2.0
## [52] gridExtra_2.3       ScaledMatrix_1.18.0 beachmat_2.26.0
## [55] evaluate_1.0.5      knitr_1.51          viridisLite_0.4.2
## [58] irlba_2.3.7         rlang_1.1.7         Rcpp_1.1.1
## [61] glue_1.8.0          R6_2.6.1
```